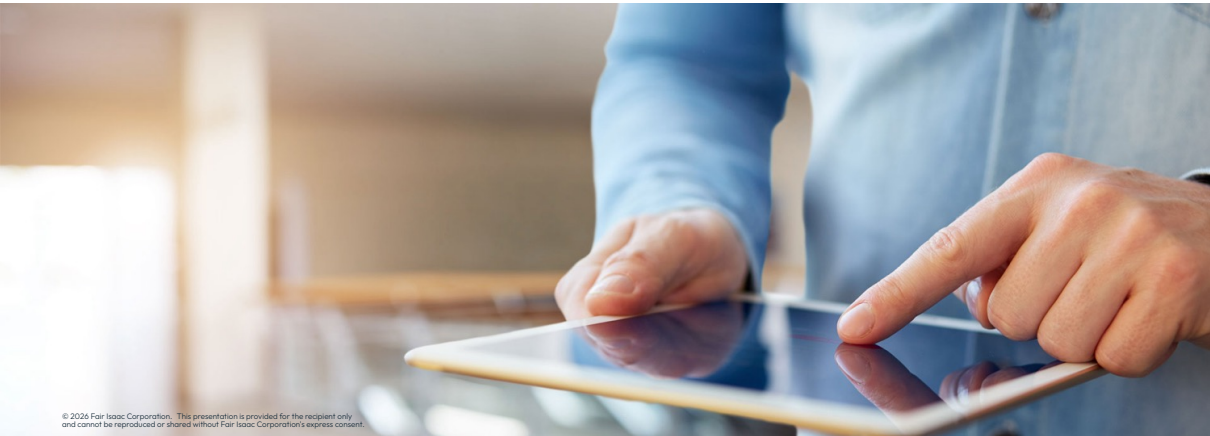
Tolerances and scaling



© 2026 Fair Isaac Corporation. This presentation is provided for the recipient only and cannot be reproduced or shared without Fair Isaac Corporation's express consent.

Tolerances and scaling

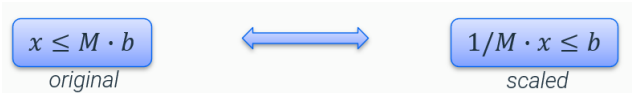
Numerical challenges

- Numerical issues arise from models with...
  - ...coefficients spanning many orders of magnitude
  - ...structures that amplify numeric error propagation
- Xpress provides tools to check coefficient ranges and condition numbers to identify numerical stability issues
- Adjust numerical tolerances, reconsider control values, and use alternative strategies like different simplex algorithms or barrier methods

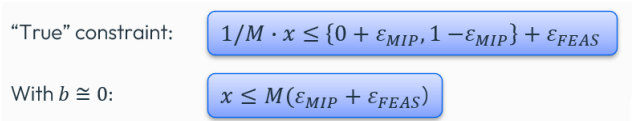
Finding help: *Check the reference manual page for more information about analyzing and handling numerical issues*

Scaling and Big-M formulations

- Example of a scaled constraint (with b binary variable)



- Valid solution: any that satisfies the (scaled) constraints within tolerances:

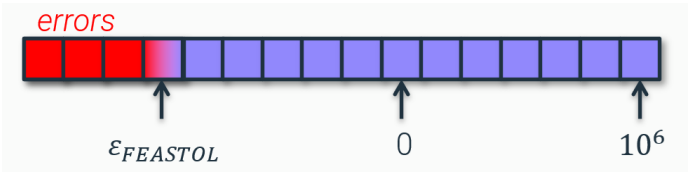


  - By default: `miptol = 5 · 10^6` and `feastol = 10^6`

- Example: $M = 10^8 \Rightarrow x \leqslant 10^8 \cdot 6 \cdot 10^{-6} = 600$ is feasible (even if $b = 0$)!

Rule of 10^6

- Double precision floating point 16-digit “budget“:



- Too large coefficients can cause round-off errors to exceed feasibility tolerance
- To stay relatively safe, use the following limits:
  - Largest value: 10^6
  - Smallest value: 10^{-6}
  - Largest row/column ratio: 10^6

Adjust numerical tolerances

- The feasibility tolerance control feastol determines when a solution is treated as feasible:

```
p.controls.feastol = 1e-6
```

  - A constraint or bound violated by less than `feastol` is treated as satisfied
- The miptol value determines when a decision variable's value is considered to be integral:

```
p.controls.miptol = 5e-6
```

  - Must be slightly larger than feastol; larger values may cause slight infeasibilities when fixing integers
- **Important:** use `feastol`/`miptol` to express the feasibility your application needs. Tightening them does not make a numerically unstable model more reliable: on ill-conditioned models it can even lead to incorrect dual bounds or false infeasibility. To address instability, use the tools on the next slides.

Scaling

- Xpress provides the user with information on the coefficient ranges in both the original problem and the problem after presolving and scaling has been applied:

```
Coefficient range                    original                    solved
  Coefficients   [min,max] : [ 1.00e+00,  8.40e+06] / [ 1.56e-01,  1.60e+01]
  RHS and bounds [min,max] : [ 1.00e+00,  8.44e+06] / [ 2.44e-02,  1.60e+01]
  Objective      [min,max] : [ 1.00e+00,  1.00e+06] / [ 4.10e+03,  2.05e+09]
Autoscaling applied Curtis-Reid scaling
```

- Autoscaling will select a scaling method that improves the coefficient distribution:

```
p.controls.autoscaling = -1
```

  - Can be disabled (0), or set to a cautious (1), moderate (2), or aggressive (3) strategy

- A more detailed scaling strategy can be defined using the bitwise scaling control:

```
p.controls.scaling = 163
```

  - If set to a non-default value by the user, autoscaling will be ignored

Diagnosing stability: the attention level

- For MIP problems, set mipkappafreq to compute the basis condition number (kappa) during the branch-and-bound search:

```
p.controls.mipkappafreq = 1
```

- The log then reports the share of bases that are stable, suspicious, unstable or ill-posed, and an overall attention level between 0 and 1:
  - 0 means all bases are stable, 1 means all are ill-posed: the higher the value, the more likely numerical errors are
  - Rule of thumb: an attention level above 0.1 should be investigated further
  - Available afterwards as the `attentionlevel` attribute (and `maxkappa` for the largest kappa sampled)
- After the root LP, Xpress also predicts the attention level from matrix features (predictedattlevel): a prediction above 0.1 logs 'High attention level predicted from matrix features'

Addressing numerical issues

- Improve the model first: narrow coefficient ranges (rule of 10^6), prefer indicators over *Big-M*
- Emphasize stability over speed with numericalemphasis (the first control to try):

```
p.controls.numericalemphasis = 1    # mild (1), medium (2), strong (3)
```

- If still hard to solve reliably, try alternative scaling, switching off dedicated presolve operations, or a different `dualstrategy`
- Refine the solution rather than loosen feasibility: the solution refiner targets higher precision after solving (see next slide)

Finding help: *See Analyzing and Handling Numerical Issues in the reference manual for the full workflow*

Solution refinement

- Xpress includes two methods for solution refinement:
  - LP Refinement (ON by default) - providing LP solutions of a higher precision:
    - iteratively attempts to increase the accuracy of the solution until tolerance targets are satisfied
  - MIP Refinement - providing MIP solutions which are truly integral:
    - attempts to round any fractional MIP entity and reduce LP infeasiblity
  - Both methods can lead to a slowdown of the solution process, which is more considerable the more numerically challenging the matrix is
- Use the refineops control to specify when the solution refiner should be executed to reduce solution infeasibilities (bit vector control):
  - The refiner will attempt to satisfy the target tolerances for all original linear constraints before presolve or scaling has been applied
- Set the precision the refiner aims for with `feastoltarget`, `miptoltarget` and `optimalitytarget`:
  - Unlike `feastol`/`miptol`, these tighten the achieved accuracy without changing what counts as feasible

Indicator constraints as alternative to *Big-M* formulations

- Indicator constraints are a very important alternative to *Big-M* formulations:
  - The solver will convert an indicator to a *Big-M* constraint if it can deduce a sufficiently small *M* that is numerically safe

- An indicator constraint is a logic constraint that expresses the implication 'if indicator condition holds then apply the constraint':
  - Represented by a tuple containing a condition on a binary variable, called the indicator, and an expression representing a constraint: `(indicator condition, constraint)`

- Indicator constraints are defined by using the problem.addIndicator() method:

```
p.addIndicator(c1, c2, ...)
```

  - Each argument `c1,c2,...` can be a single indicator constraint, or a list, tuple, or *NumPy* array of indicator constraints (tuples)
  - The constraint is only enforced when the value of the indicator variable matches a user-defined value (0 or 1)

Indicator constraints as alternative to *Big-M* formulations

- Example enforcing the constraint `y <= 15` when binary variable `x = 1` for an optimization problem `p`:

```
x = p.addVariable(vartype=xp.binary)
y = p.addVariable(lb=10, ub=20)
ind1 = (x == 1, y <= 15)
p.addIndicator(ind1)
```

Note: *The addIndicator() method also accepts nonlinear expressions for the constraint to enforce*

Python Notebook [PN-2.1]: Tolerance settings

- Navigate to Exercise 2 - Tolerances and scaling, section 2.1 (TODO 2.1)
- Run the Python code cell with the following model:

$$
\begin{aligned}
\max \quad & x_1 \\
\text{s.t.} \quad & x_1 - x_2 = 1.0000001 \\
& x_1 \leqslant 1 \\
& x_1 \geqslant 0, x_2 \geqslant 0
\end{aligned}
$$

  - Do you get the expected result?
  - Which tolerance's value would you need to change to make this work?

Python Notebook [PN-2.2]: MIP tolerance settings

- Navigate to Exercise 2 - Tolerances and scaling, section 2.2 (TODO 2.2)
- Run the Python code cell with the following model:

$$
\begin{aligned}
\min \quad & y \\
\text{s.t.} \quad & x \leqslant 1000000 \cdot y \\
& x \geqslant 1000001 \\
& y \text{ integer}
\end{aligned}
$$

  - Is the solution correct? Why ?
  - What happens after we reduce the MIP tolerance and adjust the feasibility tolerance ?

Thank You

www.fico.com/optimization